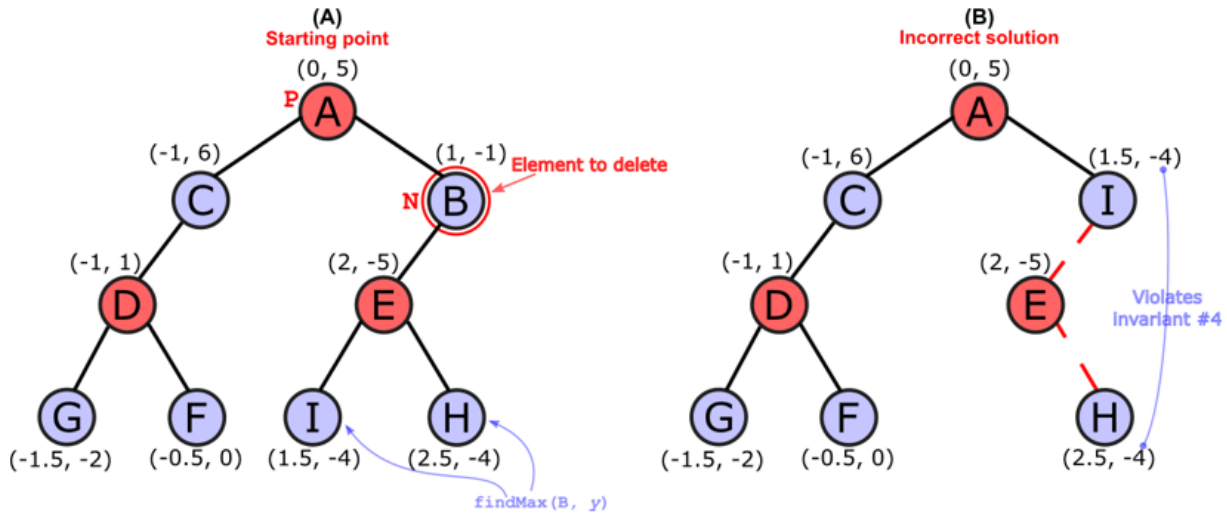


Figure 9.18 An example showing why, when we delete a node with only a left child, we can't replace current node with the minimum of the left branch. On the right we can see how `H` causes the 4th invariant of k-d trees to be violated, because it is on the left branch of node `I`, but has the same value for `I`'s split coordinate.

Luckily the solution is not too complicated: we can instead run `findMin` on the left branch, replace `N`'s point with the node `M` found by `findMin`, and set `N`'s old left branch as the right branch of this new node we are creating, like we show in figure 9.19.

Then we just need to remove the old node `M` from that right branch: notice that, differently from what happened with binary search trees, we can make no assumptions on `M` here, so we might need to repeat these steps in the recursive call deleting `M`.

If we look at the running time for `remove`, the cost of the calls to `findMin` drives up the total cost, that thus can't be